Transformers in the World

The family, the LM head, and what it costs

Before we start: videos and a demo

Recommended viewing

Hugging Face's course, three short videos (16 min in all): Transformer models: Encoders, Transformer models: Decoders, Transformer models: Encoder-Decoders. Today's first section.

3Blue1Brown, Transformers, the tech behind LLMs | Deep Learning Chapter 5, from 20:22: the unembedding, then softmax with temperature (22:22). Today's second section, and LLM-6's opening.

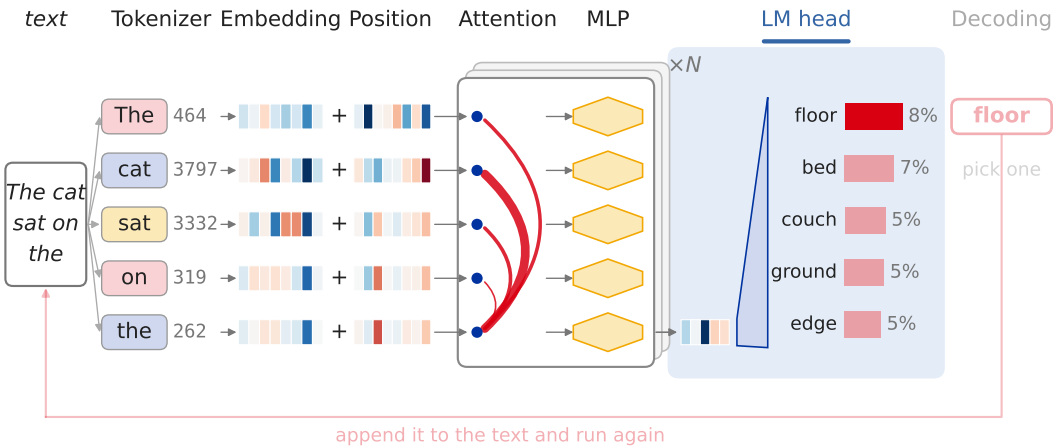
Live demo: bbycroft.net/llm

Brendan Bycroft's LLM Visualization: every matrix of a transformer in 3D. A guided walk through a tiny model, then GPT-2 small, GPT-2 XL and GPT-3, drawn to scale.

Try:

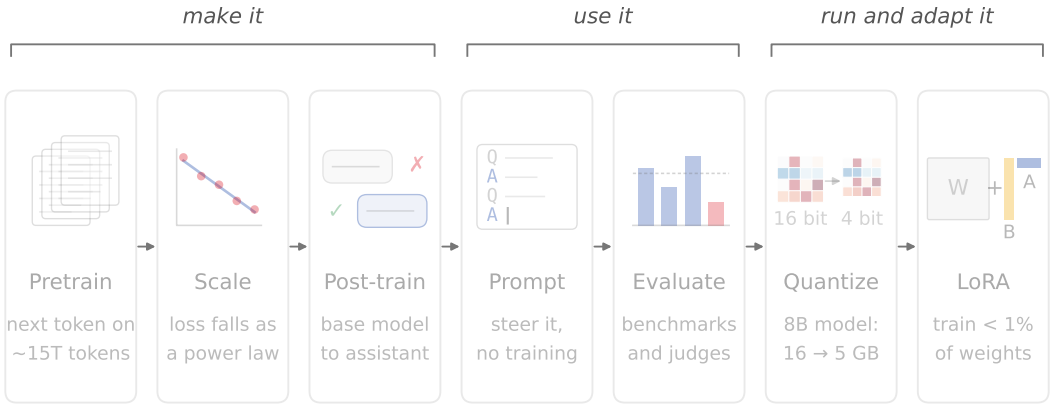
- the walk-through, up to "Output"
- GPT-2 small next to GPT-3: the same block, bigger
- find the LM head at the very end

Where we are: one forward pass



Today: the **LM head** (last vector to a distribution) and three ways to wire the block.

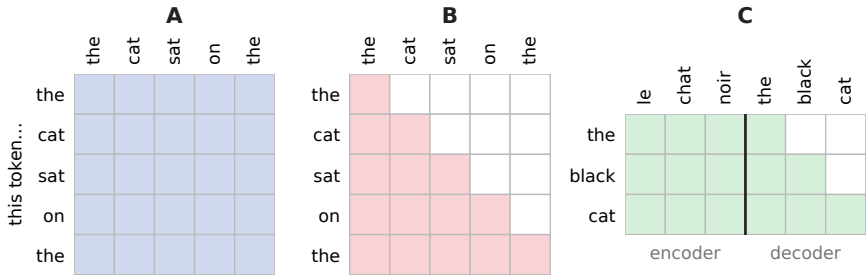
Where we are: the life of a model



Later: where these weights come from, and what you do with them once you have them.

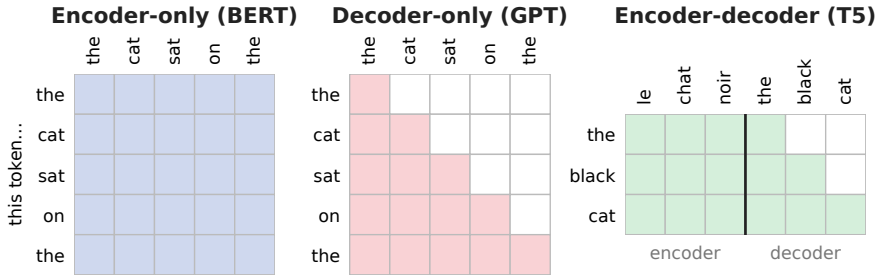
Which of these can write text?

Three attention masks for the same block. A row is the token being computed; the shaded cells are the tokens it may look at.



Predict first: which of A, B, C could *write* a sentence, left to right, one token at a time? More than one answer is allowed.

B and C can. A cannot.



A lets the first "the" look at "sat". When you write, word 3 does not exist yet while you choose word 2, so a model trained to peek cannot be run forwards. **B** is LLM-4's causal mask. **C** writes English with a causal mask too, and also reads the whole French sentence (drawn as one grid; in the model it is two attention layers, one over each sequence).

The block is the same in all three. The difference is one decision: **who is allowed to look at whom?**

Outline

The family

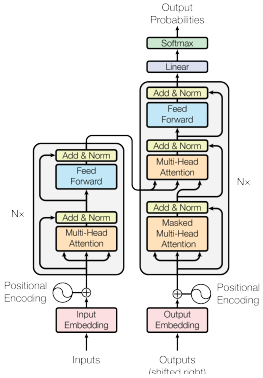
The LM head

Why it won, and what it costs

The family

One block, three ways of wiring it.

The original: an encoder and a decoder



Vaswani et al. (2017), Figure 1

The 2017 paper was about **translation**, so it has two stacks:

Encoder – reads the source sentence. Every token may see every other token.

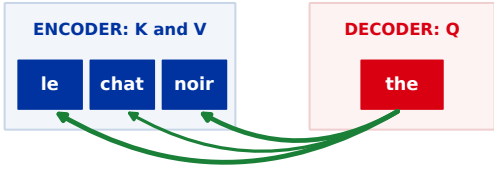
Decoder – writes the target sentence, one token at a time, so it is **causally masked** (LLM-4).

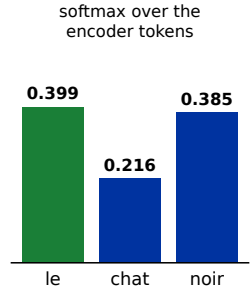
They are joined by **cross-attention**: the decoder supplies the queries, and the *encoder's* output supplies the keys and values.

Self-attention asks "what in *this* sentence matters to me?" Cross-attention asks "what in *that other* sequence matters to me?" Identical mechanism; the keys and values just come from somewhere else.

Cross-attention, with actual numbers

The decoder is writing an English sentence and has produced "the". The encoder has already read the French source. Where should "the" look?





One head, $d_k = 4$, toy random weights. Each score is $q \cdot k/\sqrt{4}$: for "le", $0.6788/2 = 0.3394 \approx +0.339$; the others are -0.273 and $+0.304$. Softmax gives **0.399**, **0.216**, **0.385** (sum 1), and the output is $0.399\, v_{\text{le}} + 0.216\, v_{\text{chat}} + 0.385\, v_{\text{noir}}$, exactly as in LLM-3.

What actually changed

Compare with the self-attention you computed in LLM-3. The formula is character-for-character the same:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

	Self-attention	**Cross-attention**
Q comes from	this sequence	**the decoder**
K, V come from	this sequence	**the encoder**

Where it sits: every decoder block has **three** sublayers - masked self-attention, then cross-attention (its own W_Q, W_K, W_V; the keys and values come from the encoder's *final* output, the same for every decoder block), then the MLP.

That is the entire difference: **two of the three inputs come from somewhere else**. No new mechanism, no new maths – which is why the original paper could reuse the same block for both.

Three architectures



same block, three ways of deciding who may look at whom

Nobody changed the block. **Encoder-only** sees in both directions and is good at understanding: BERT is trained to fill in hidden words, with both sides visible. **Decoder-only** sees only backwards, which makes it able to generate. **Encoder-decoder** keeps both and joins them with cross-attention.

The family, by name

- **Encoder-only**: BERT (Google, 2018). Not retired: ModernBERT (December 2024) was trained on 2 trillion tokens, reads 8,192 tokens at once, and is built for classification and retrieval - the embeddings that retrieval-augmented generation (RAG) needs.
- **Encoder-decoder**: T5 (Raffel et al., 2020) turns every task into text in, text out, with a prefix that names the task: "translate English to German: That is good." $\rightarrow$ "Das ist gut."; "cola sentence: John made Bill master of himself." $\rightarrow$ "acceptable" (cola: is the sentence grammatical?).
- **Decoder-only**: GPT-2, and every chat model you have used.

A running joke of 2018-2020: ELMo (2018), BERT (2018), ERNIE (2019), Grover (2019), KERMIT (2019), Big Bird (2020). For a while, language models were named after Sesame Street.

Why decoder-only won the last five years

- **One objective, unlimited data.** Predicting the next token needs no labels, so any text on the internet is training data.
- **It subsumes the others.** "Translate this: ..." is just a prompt. Tasks that needed a separate architecture became a formatting question.
- **It is the simplest thing to scale.** One stack, one loss, no encoder to keep in sync.

This is a statement about the last few years, not a theorem. Encoder-only models are still the right tool when you want an *embedding* rather than a continuation – which is exactly what the retrieval chapter (RAG, after LLM-11) will use them for.

Mask A writes after all: diffusion language models

- The cold open: mask A cannot write *left to right*. Diffusion language models do not write left to right. They start from an answer that is all blanks, predict every blank at once, keep the confident guesses, blank out the rest again, and repeat for a fixed number of steps.
- LLaDA (Nie et al., NeurIPS 2025): 8B parameters, "does not use a causal mask", trained from scratch on 2.3 trillion tokens; "competitive with" LLaMA 3 8B.
- The selling point is speed, since many tokens are filled per step: Inception's Mercury (2025) over 1,000 tokens per second on one GPU; Google's open DiffusionGemma (June 2026) up to 4× faster than writing token by token, with quality below Gemma 4.

The cold open's answer holds for writing one token at a time. Write everything at once and refine it, and mask A works - so far trading some quality for speed.

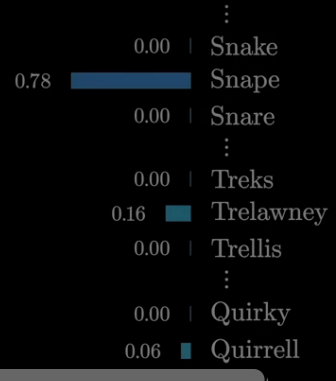
Nie et al., arXiv 2502.09992; Inception Labs (Feb 2025); Google (10 June 2026). Gemini Diffusion (May 2025) is still an experimental demo.

The LM head

From the last vector to a probability for every token.

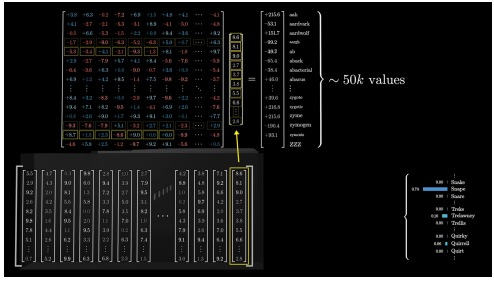
Harry Potter was a highly unusual boy ... least favourite teacher, Professor ???





Next-token prediction, in practice: the model outputs a distribution over the whole vocabulary, and one continuation is far more likely than the rest.

From a vector to a word



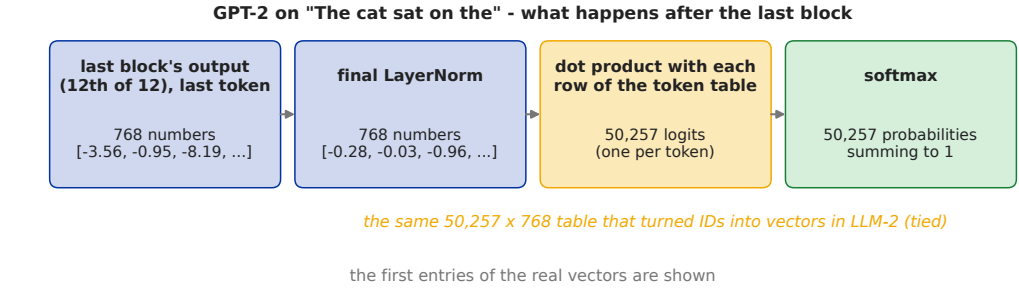
3Blue1Brown

Every position leaves the last block with a vector. In training, every one of them predicts its own next token at once (LLM-4's mask keeps that honest). When generating, the earlier next tokens are already written, so only the **last** vector's prediction is new. After one final LayerNorm it is multiplied by an **unembedding matrix** – one row per vocabulary entry, so roughly 50,000 numbers come out.

Those are **logits**. A softmax turns them into a probability distribution, and you sample from it. That is the entire output stage.

The model produces a *distribution* over every word, not a word. Choosing is a separate decision: temperature and the other decoding strategies (LLM-6).

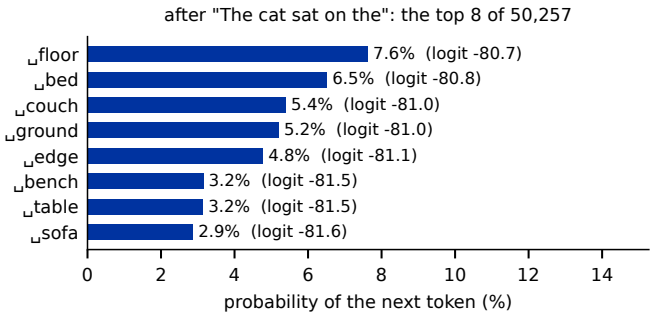
GPT-2's head, on the map's prompt



The "unembedding matrix" in GPT-2 is not a new matrix. It is the token table from LLM-2, used a second time: **tied**. Row j turned token j into a vector on the way in; on the way out, its dot product with the last vector scores token j. Both jobs want tokens used alike to get vectors alike (LLM-2).

The table is $50{,}257 \times 768 = 38.6\text{M}$ numbers, **31%** of GPT-2's 124.4M parameters. Tying it means the head costs no parameters of its own.

What comes out



Real output of the cached GPT-2 small. (␣ is the leading space, part of the token: LLM-1.)

The top five hold **30%** of the probability. Only **14** of the 50,257 tokens get more than 1%; the remaining mass is spread thinly over everything else.

And look at the logits: **all of them are near** -80.

Predict: if every logit is around -80, how can "floor" end up with 7.6%?

One logit, by hand

A logit is one dot product, 768 terms long: the last vector against one row of the token table. For the token "floor":

$$z(\text{floor}) = (-0.284)(-0.122) + (-0.033)(0.161) + (-0.962)(-0.025) + (0.225)(-0.024) + \dots$$

The first four terms give 0.048; all 768 give $z(\text{floor}) = -80.65$. Likewise $z(\text{bed}) = -80.81$.

Every row of the table gets its own dot product: 50,257 of them, which is one matrix multiplication. A row that points the same way as the last vector gets a high score.

Every logit is near -80. Back to the question: how does "floor" end up with 7.6%?

Answer: only differences matter

Two tokens: compare them directly.

$$\frac{p(\text{floor})}{p(\text{bed})} = \frac{\exp z(\text{floor})}{\exp z(\text{bed})} = \exp\big(z(\text{floor}) - z(\text{bed})\big) = \exp(0.157) = 1.17$$

Check against the bars: $7.64\% \,/\, 6.53\% = 1.17$.

One token: compare it with all of them.

$$p(\text{floor}) = \frac{\exp z(\text{floor})}{\sum_j \exp z_j} = \exp\big(z(\text{floor}) - \log \textstyle\sum_j \exp z_j\big) \qquad (j \text{ runs over all 50,257 tokens})$$

$$= \exp\big(-80.654 - (-78.082)\big) = \exp(-2.572) = \mathbf{7.64\%}$$

$\log \sum_j \exp z_j = -78.08$ sits *above* the largest logit (-80.65): 50,257 positive terms add up to more than the largest one alone.

Add the same number c to every logit and nothing changes: $e^{z_i+c} / \sum_j e^{z_j+c} = e^{z_i} / \sum_j e^{z_j}$, because the factor e^c cancels. The softmax only sees **differences**. That is why -80 is harmless, and it is the knob LLM-6 turns: temperature stretches or shrinks those differences.

Why it won, and what it costs

Every architecture buys something with something.

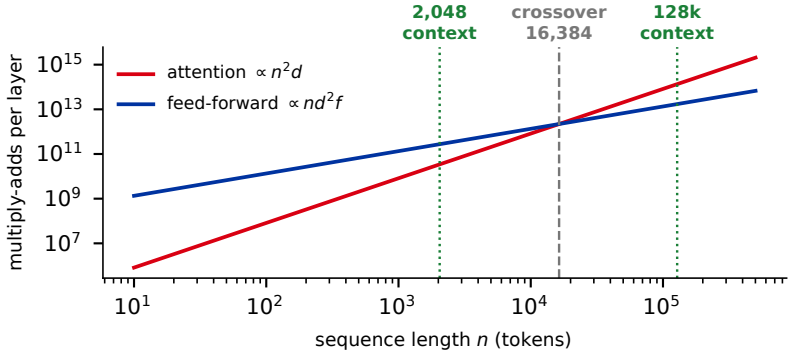
Why transformers won

1. **It parallelises.** An RNN had to walk the sequence in order. Attention is a few big matrix multiplications over the whole sequence at once (LLM-3), which is exactly the workload a GPU wants. That is training; generating is still one token at a time (LLM-6).
2. **Because it parallelises, it scales.** You can afford more data and more parameters.
3. **Because it scales, it kept getting better.** The clearest example is **in-context learning**: give a model a few examples in the prompt and it follows the pattern, *with no weight updates at all*. Small models do it badly, large ones well; whether that improvement is sudden or gradual is debated (LLM-8).

Notice the shape of that argument: the win is not that attention is a cleverer model of language. It is that attention is a **better fit for the hardware**, and everything else followed from being able to train bigger. Architecture that suits the machine beats architecture that suits your intuition.

The bill: attention is quadratic

Every token attends to every token, so the score matrix is $n \times n$. Double the context and you quadruple that cost.



$d_{\text{model}} = 4096$, $f = 4$. The grid costs memory too: at 128,000 tokens one head's $n \times n$ scores are 16.4 billion numbers, so FlashAttention never stores it whole (LLM-14).

Read that graph carefully

The usual story is "attention is the bottleneck". The arithmetic says: **not until the sequence gets long**.

You can work out where it flips in two lines. Attention's $n \times n$ part costs about $n^2 d$ (its Q, K, V and output projections grow linearly in n, so they count with the feed-forward); the feed-forward costs about $nd^2 f$, where f is the widening factor (usually 4). Set them equal:

$$n^2 d = nd^2 f \quad \Longrightarrow \quad \boxed{n = d \cdot f}$$

So for $d_{\text{model}} = 4096$ and $f = 4$, the crossover is at $4096 \times 4 = \mathbf{16{,}384}$ tokens. Halve the model and you halve the crossover. For GPT-2 small, $768 \times 4 = 3{,}072$: three times its whole 1,024-token window.

So for a 2,048-token context, optimising attention is optimising the smaller half. "Quadratic" describes how the cost *grows*, not where the cost *is* at every size. FlashAttention and sparse variants attack the n^2 term; 2025 hybrids keep full attention in a quarter of their layers or fewer (LLM-14).

The honest limit: almost no inductive bias

Every architecture in this course encoded an assumption about its data:

Model	Built-in assumption	Consequence
Decision tree	axis-aligned splits matter	works on tiny tabular data
CNN	nearby pixels are related	learns vision from thousands of images
RNN	order matters, recent matters more	handles sequences by construction
Transformer	**almost nothing**	**needs enormous data**

A Transformer does not know that neighbouring words are related, or even that words have an order – we had to *bolt position on* in LLM-2.

That weakness is the same fact as its strength. Assuming nothing means it can learn any structure, and **must** learn all of it from data. This is why a ViT loses to a CNN on a small image dataset and wins on a huge one (Dosovitskiy et al., 2021: a few points below ResNets on ImageNet alone, ahead after 14-300M images), and why transformers arrived only once the data and the compute did.

Same block, everywhere

Domain	What became a token	Chapter
Text	subword pieces	this chapter
Images (ViT)	16×16 patches	vision-language models
Audio	spectrogram frames or codec codes	audio
Proteins	amino acids	protein language models, AlphaFold
Actions	robot commands	vision-language-action

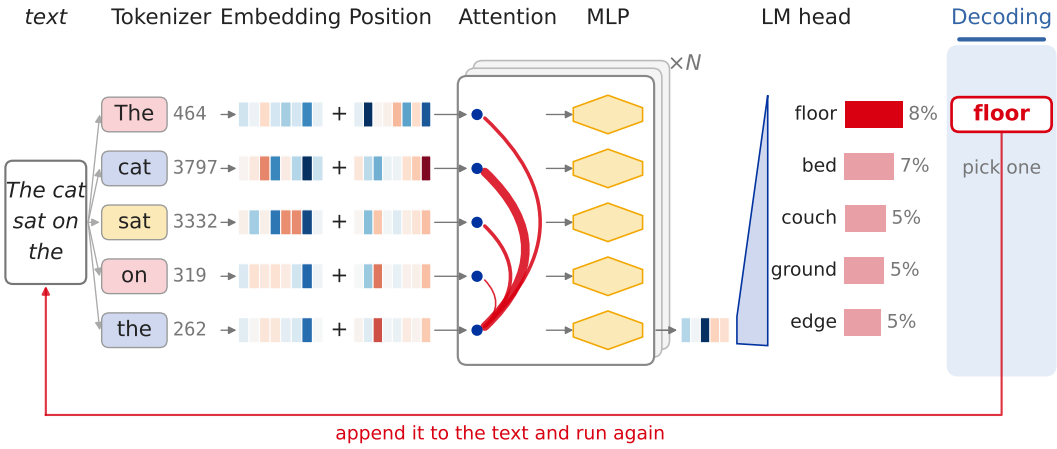
Nobody redesigned the block for any of these. They redesigned what counts as a **token**. That is the practical legacy of the architecture: if you can serialise your problem into a sequence of vectors, you can throw a Transformer at it.

AlphaFold 2's core (the Evoformer, 48 blocks) is attention over protein sequences and pairs of residues. Half of the 2024 Nobel Prize in Chemistry went to Demis Hassabis and John Jumper "for protein structure prediction".

Recap

- **One block, three wirings.** Encoder-only (BERT) understands, decoder-only (GPT) generates, encoder-decoder (T5) translates. The difference is who may look at whom - the mask, plus cross-attention in the decoder - not the attention itself.
- **Cross-attention** is ordinary attention with the keys and values taken from another sequence. Diffusion LMs write with no causal mask at all, many tokens per step.
- **The LM head** is one dot product per vocabulary entry with the token table (tied in GPT-2), then a softmax. The output is a distribution, not a word, and only the differences between logits matter.
- **It won because it fits the GPU**, not because it is a better theory of language.
- **It is quadratic in sequence length**, but at $d_{\text{model}} = 4096$ that only dominates past 16,384 tokens.
- **It assumes almost nothing**, so it can learn anything and needs vast data to do it.

Next on the map



Next (LLM-6): **decoding** - how a distribution becomes the text you read.

Sources & further reading

- **Watch:** Hugging Face course (2021), "Transformer models: Encoders / Decoders / Encoder-Decoders"; 3Blue1Brown (2024), Deep Learning Chapter 5. **Play:** bbycroft.net/llm.
- Vaswani et al. (2017), "Attention Is All You Need", NeurIPS (the encoder-decoder, cross-attention).
- Devlin et al. (2019), "BERT", NAACL; Raffel et al. (2020), "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer" (T5), JMLR; Warner et al. (2025), "ModernBERT", ACL.
- Radford et al. (2019), "Language Models are Unsupervised Multitask Learners" (GPT-2).
- Nie et al. (2025), "Large Language Diffusion Models" (LLaDA), NeurIPS.
- Dosovitskiy et al. (2021), "An Image is Worth 16x16 Words" (ViT), ICLR; Jumper et al. (2021), "Highly accurate protein structure prediction with AlphaFold", *Nature*.
- Qwen team (2025), Qwen3-Next model card; Moonshot AI (2025), "Kimi Linear"; NVIDIA (2025), "Nemotron-H".

GPT-2's head: `py_src/head_figs.py` (cached GPT-2 small, raw results in `results/head_gpt2.json`).